
Projet – Orchestration automatisée globale

Partie 1 — Cadrage & architecture

L'entreprise souhaite garantir la continuité de son application conteneurisée en s'appuyant sur deux plateformes complémentaires : Amazon ECS pour la production cloud managée, et Kubernetes pour la portabilité et l'indépendance vis-à-vis d'un fournisseur unique. L'objectif est d'industrialiser ces deux déploiements via une chaîne d'automatisation unique et reproductible (Git, Terraform, Jenkins), afin d'éliminer toute étape manuelle non tracée et de pouvoir recréer l'environnement à l'identique en cas de panne ou de migration.

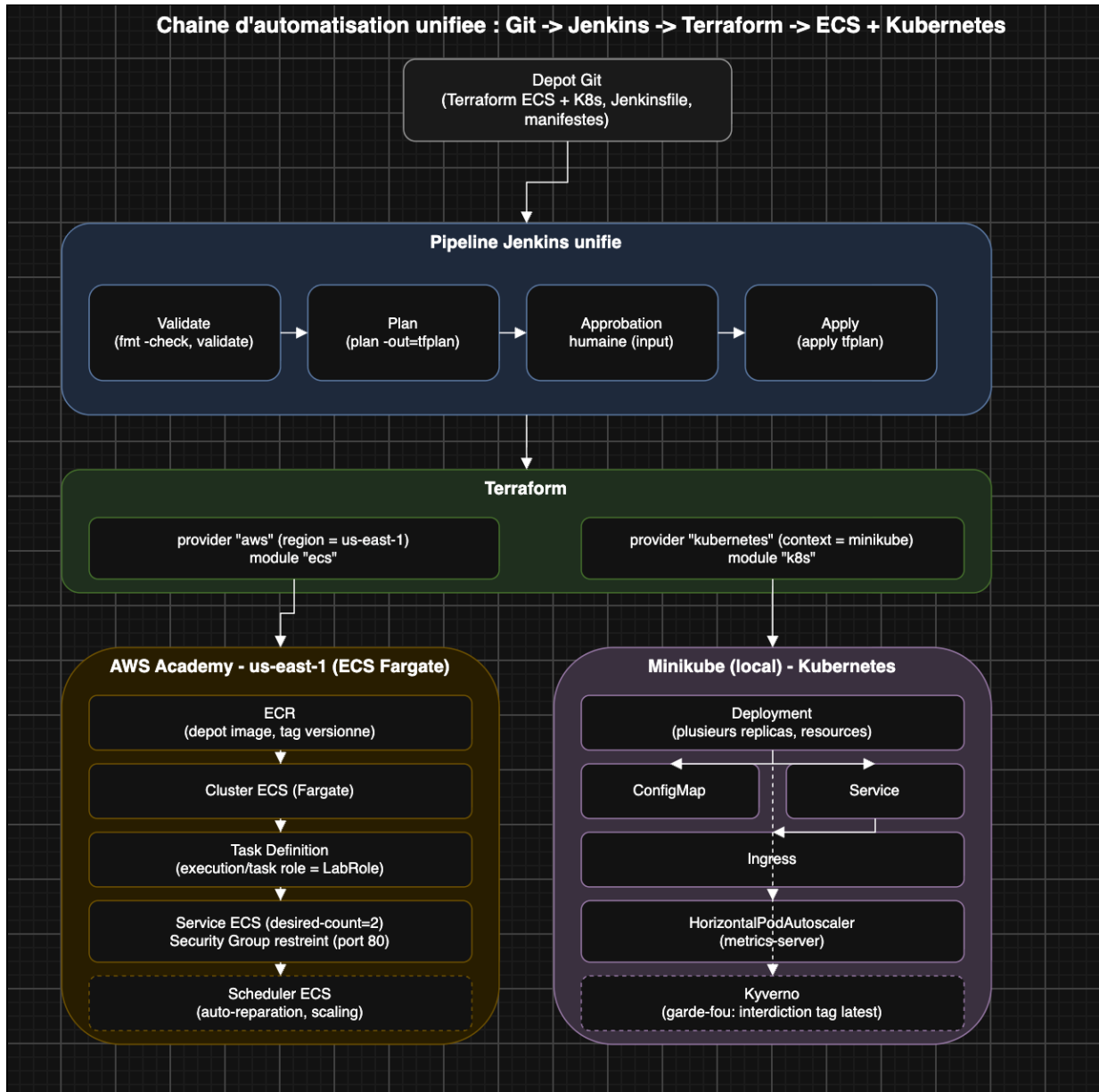
En termes de **charge attendue**, l'application cible un trafic modéré mais variable : le dimensionnement retenu est de deux instances au repos sur chaque plateforme (deux tâches Fargate côté ECS, deux répliques côté Kubernetes), avec une capacité de montée en charge automatique jusqu'à cinq unités en cas de pic, pilotée respectivement par le scheduler ECS (desired-count) et le HorizontalPodAutoscaler basé sur l'utilisation CPU.

Concernant la **disponibilité**, la résilience est assurée par la redondance des instances et l'auto-réparation propre à chaque orchestrateur : ECS relance automatiquement toute tâche défaillante détectée par le scheduler, tandis que Kubernetes s'appuie sur les sondes de liveness et le contrôleur de Deployment pour recréer les pods en échec, garantissant qu'aucune coupure ne dépende d'une intervention humaine.

Sur le plan de la **sécurité**, le principe de moindre privilège est appliqué avec le rôle LabRole imposé côté ECS (aucune création de rôle IAM autorisée), un Security Group restreint au port applicatif nécessaire, et côté Kubernetes une politique Kyverno interdisant l'usage du tag latest sur les images pour garantir la traçabilité des versions déployées ; aucun secret n'est stocké en clair dans le dépôt Git.

Enfin, côté **coût**, le choix de Fargate évite l'exploitation d'instances EC2 dédiées et facture uniquement les ressources consommées par les tâches actives, tandis que Kubernetes est testé localement via Minikube pour éviter tout coût de cluster managé pendant la phase de développement, les manifestes restants directement transposables vers EKS en production.

Schéma de l'architecture cible :



Partie 2 — Déploiement sur ECS via Terraform

```
charbel@charbzz-macbook-air terraform % terraform init
```

Initializing the backend...

Initializing modules...

- k8s in modules/k8s
- ecs in modules/ecs

Initializing provider plugins...

- Finding hashicorp/kubernetes versions matching "~> 2.25"...
- Finding hashicorp/aws versions matching "~> 5.0"...
- Installing hashicorp/kubernetes v2.38.0...
- Installed hashicorp/kubernetes v2.38.0 (signed by HashiCorp)
- Installing hashicorp/aws v5.100.0...
- Installed hashicorp/aws v5.100.0 (signed by HashiCorp)

Terraform has created a lock file **.terraform.lock.hcl** to record the provider selections it made above. Include this file in your version control repository so that Terraform can guarantee to make the same selections by default when you run "terraform init" in the future.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
charbel@charbzz-macbook-air terraform % terraform validate
```

Success! The configuration is valid.

```
charbel@charbzz-macbook-air terraform % terraform apply tfplan-ecs
```

```
module.ecs.aws_ecr_repository.app: Creating...
module.ecs.aws_ecs_cluster.main: Creating...
module.ecs.aws_security_group.svc_sg: Creating...
module.ecs.aws_ecr_repository.app: Creation complete after 1s [id=demo-app]
module.ecs.aws_ecs_task_definition.app: Creating...
module.ecs.aws_security_group.svc_sg: Creation complete after 4s [id=sg-0666dfc12bf61378d]
module.ecs.aws_ecs_cluster.main: Still creating... [00m10s elapsed]
module.ecs.aws_ecs_cluster.main: Creation complete after 12s [id=arn:aws:ecs:us-east-1:661688898602:cluster/demo-cluster]
module.ecs.aws_ecs_service.app: Creating...
module.ecs.aws_ecs_service.app: Creation complete after 1s [id=arn:aws:ecs:us-east-1:661688898602:service/demo-cluster/demo-app-service]
```

Warning: Applied changes may be incomplete

The plan was created with the `-target` option in effect, so some changes requested in the configuration may have been ignored and the output values may not be fully updated. Run the following command to verify that no other changes are pending:

```
terraform plan
```

Note that the `-target` option is not suitable for routine use, and is provided only for exceptional situations such as recovering from errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Apply complete! Resources: 5 added, 0 changed, 0 destroyed.

Outputs:

```
ecr_repository_url = "661688898602.dkr.ecr.us-east-1.amazonaws.com/demo-app"
```

BOU HANNA
Charbel
MCS5 26.4
<https://github.com/charbel-bh/orchestration-ecs-k8s>

Belkacem
Maaref
MCS5 26.4

```
charbel@charbzz-macbook-air terraform % docker push $REPO_URL:1.0
```

```
The push refers to repository [661688898602.dkr.ecr.us-east-1.amazonaws.com/demo-app]
62603bffffbf3: Pushed
44136fa355b3: Pushed
61ca4f733c80: Pushed
f18232174bc9: Pushed
b464cfd2a63: Pushed
d7e507024086: Pushed
81bd8ed7ec67: Pushed
34a64644b756: Pushed
197eb75867ef: Pushed
39c2ddfd6010: Pushed
57f695c337e6: Pushed
1.0: digest: sha256:64b994df5286c446d17d68e0bbf6cfe82c9aacfda6ece2ddf5b1c3dc7c6c35dc size: 856
```

```
charbel@charbzz-macbook-air terraform % curl http://$PUBLIC_IP
<h1>Orchestration ECS + Kubernetes – OK</h1>
```

```
[charbel@charbzz-macbook-air terraform % aws ecs stop-task --cluster demo-cluster --task $TASK_ARN --region us-east-1
{
  "task": {
    "attachments": [],
    "availabilityZone": "us-east-1f",
    "clusterArn": "arn:aws:ecs:us-east-1:661688898602:cluster/demo-cluster",
    "connectivity": "CONNECTED",
    "connectivityAt": "2026-08-28T12:37:26.256000+02:00",
    "containers": [],
    "cpu": "256",
    "createdAt": "2026-08-28T12:37:22.468000+02:00",
    "desiredStatus": "STOPPED",
    "enableExecuteCommand": false,
    "executionStoppedAt": "2026-08-28T12:40:14.623000+02:00",
    "group": "service:demo-app-service",
    "lastStatus": "STOPPED",
    "launchType": "FARGATE",
    "memory": "512",
    "overrides": {
      "containerOverrides": [],
      "inferenceAcceleratorOverrides": []
    },
    "platformVersion": "1.4.0",
    "platformFamily": "Linux",
    "pullStartedAt": "2026-08-28T12:37:34.043000+02:00",
    "pullStoppedAt": "2026-08-28T12:37:41.142000+02:00",
    "startedAt": "2026-08-28T12:37:41.357000+02:00",
    "startedBy": "ecs-svc/9011829400580940920",
    "stopCode": "UserInitiated",
    "stoppedAt": "2026-08-28T12:40:38.028000+02:00",
    "stoppedReason": "Task stopped by user",
    "stoppingAt": "2026-08-28T12:40:13.881000+02:00",
    "tags": [],
    "taskArn": "arn:aws:ecs:us-east-1:661688898602:task/demo-cluster/35dfc12552f64308b63fcd85005e8ec",
    "taskDefinitionArn": "arn:aws:ecs:us-east-1:661688898602:task-definition/demo-app-task:1",
    "version": 7,
    "ephemeralStorage": {
      "sizeInGiB": 20
    },
    "fargateEphemeralStorage": {
      "sizeInGiB": 20
    }
  }
}
```

```
charbel@charbzz-macbook-air terraform % watch aws ecs list-tasks --cluster demo-cluster --region us-east-1
```

```
Every 2.0s: aws ecs list-tasks --cluster demo-cluster --region us-east-1 charbzz-macbook-air.local: ven. 28 août 12:53:37 2026
in 0.612s (0)
{
  "taskArns": [
    "arn:aws:ecs:us-east-1:661688898602:task/demo-cluster/4d2acc549c7c490db923f0e6550256a",
    "arn:aws:ecs:us-east-1:661688898602:task/demo-cluster/f2a69d20ad3b4041af0e8493e15f5210"
  ]
}
```


Partie 3 — Déploiement sur Kubernetes via Terraform

```
charbel@charbzz-macbook-air terraform % terraform apply tfplan-k8s
module.k8s.kubernetes_config_map.app_config: Creating...
module.k8s.kubernetes_service.app: Creating...
module.k8s.kubernetes_deployment.app: Creating...
module.k8s.kubernetes_config_map.app_config: Creation complete after 0s [id=default/demo-app-config]
module.k8s.kubernetes_service.app: Creation complete after 2s [id=default/demo-app-svc]
module.k8s.kubernetes_deployment.app: Creation complete after 2s [id=default/demo-app]
module.k8s.kubernetes_ingress_v1.app: Creating...
module.k8s.kubernetes_ingress_v1.app: Creation complete after 0s [id=default/demo-app-ingress]

Warning: Applied changes may be incomplete

The plan was created with the -target option in effect, so some changes requested in the configuration may have been ignored and the output values may not be fully updated. Run the following command to verify that no other changes are pending:
    terraform plan

Note that the -target option is not suitable for routine use, and is provided only for exceptional situations such as recovering from errors or mistakes, or when Terraform specifically suggests to use it as part of an error message.

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.

Outputs:
ecr_repository_url = "661688898602.dkr.ecr.us-east-1.amazonaws.com/demo-app"
```

```
charbel@charbzz-macbook-air terraform % kubectl get deployments,svc,ingress
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/demo-app	2/2	2	2	3m45s

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
service/demo-app-svc	ClusterIP	10.101.138.210	<none>	80/TCP	3m45s
service/kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	2d

NAME	CLASS	HOSTS	ADDRESS	PORTS	AGE
ingress.networking.k8s.io/demo-app-ingress	nginx	demo-app.local	192.168.49.2	80	3m43s

```
charbel@charbzz-macbook-air terraform % kubectl get pods -o wide
```

NAME	READY	STATUS	RESTARTS	AGE	IP	NODE	NOMINATED NODE	READINESS GATES
demo-app-d7df586f7-kgcrd	1/1	Running	0	3m55s	10.244.0.65	minikube	<none>	<none>
demo-app-d7df586f7-thgjb	1/1	Running	0	3m55s	10.244.0.64	minikube	<none>	<none>

```
[charbel@charbzz-macbook-air ~ % curl http://demo-app.local
<h1>Orchestration ECS + Kubernetes - OK</h1>
```

```
charbel@charbzz-macbook-air ~ % kubectl run -it load-generator --image=busybox --restart=Never -- \
[ /bin/sh -c "while true; do wget -q -O- http://demo-app-svc; done"
```

```
charbel@charbzz-macbook-air terraform % kubectl get hpa -w
```

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS	AGE
demo-app-hpa	Deployment/demo-app	cpu: 1%/50%	2	5	2	25s
demo-app-hpa	Deployment/demo-app	cpu: 38%/50%	2	5	2	3m
demo-app-hpa	Deployment/demo-app	cpu: 65%/50%	2	5	2	4m
demo-app-hpa	Deployment/demo-app	cpu: 65%/50%	2	5	3	4m15s
demo-app-hpa	Deployment/demo-app	cpu: 46%/50%	2	5	3	5m
demo-app-hpa	Deployment/demo-app	cpu: 44%/50%	2	5	3	6m

```
charbel@charbzz-macbook-air ~ % kubectl delete pod load-generator
pod "load-generator" deleted from default namespace
```

demo-app-hpa	Deployment/demo-app	cpu: 1%/50%	2	5	3	11m
demo-app-hpa	Deployment/demo-app	cpu: 1%/50%	2	5	2	12m

```
charbel@charbzz-macbook-air docs % kubectl get clusterpolicy
```

NAME	ADMISSION	BACKGROUND	READY	AGE	MESSAGE
disallow-latest-tag	true	true	True	13s	Ready

Partie 4 — Automatisation unifiée avec Jenkins

```
charbel@charbzz-macbook-air orchestration-ecs-k8s % docker logs jenkins

Running from: /usr/share/jenkins/jenkins.war
webroot: /var/jenkins_home/war
2026-08-28 13:51:25.544+0000 [id=1] INFO winstone.Logger#logInternal: Beginning extraction from war file
2026-08-28 13:51:25.567+0000 [id=1] WARNING o.e.j.ee9.nested.ContextHandler$setContextPath: Empty contextPath
2026-08-28 13:51:25.888+0000 [id=1] INFO org.eclipse.jetty.server.Server$doStart: Jetty-12.1.8; built: 2026-04-01T01:09:18.991Z; git: c9cdc9aaa43a3665b8a53b4d1cc3684992da649; jvm 21.0.11+10-LTS
2026-08-28 13:51:26.047+0000 [id=1] INFO o.e.j.e.w.StandardDescriptorProcessor$visitServlet: NO JSP Support for /, did not find org.eclipse.jetty.ee9.jsp.JettyJspServlet
2026-08-28 13:51:26.062+0000 [id=1] INFO o.e.j.s.DefaultSessionIdManager$doStart: Session workerName=node0
2026-08-28 13:51:26.281+0000 [id=1] INFO Hudson.WebAppMain#contextInitialized: Jenkins home directory: /var/jenkins_home found at: EnvVars.masterEnvVars.get("JENKINS_HOME")
2026-08-28 13:51:26.241+0000 [id=1] INFO o.e.j.s.handler.ContextHandler$doStart: Started oeje9n.ContextHandler$CoreContextHandler@39c385d6{Jenkins v2.568.2,,bfile:///var/jenkins_home/war/,a=AVAILABLE}
2026-08-28 13:51:26.247+0000 [id=1] INFO o.e.j.s.server.AbstractConnector$doStart: Started oejs.ServerConnector@859ea42{HTTP/1.1, (http/1.1)}{0.0.0.0:8080}
2026-08-28 13:51:26.247+0000 [id=1] INFO org.eclipse.jetty.server.Server$doStart: Started oejs.ServerConnector@859ea42{HTTP/1.1, (http/1.1)}{0.0.0.0:8080} [359ms]
2026-08-28 13:51:26.248+0000 [id=41] INFO winstone.Logger#logInternal: Winstone Servlet Engine running: controlPort=disabled
2026-08-28 13:51:26.288+0000 [id=40] INFO Jenkins.model.Jenkins#<init>: Starting version 2.568.2
2026-08-28 13:51:26.316+0000 [id=49] INFO Jenkins.InitReactorRunner$1#onAttained: Started initialization
2026-08-28 13:51:26.321+0000 [id=66] INFO Jenkins.InitReactorRunner$1#onAttained: Listed all plugins
2026-08-28 13:51:26.631+0000 [id=66] INFO Jenkins.InitReactorRunner$1#onAttained: Prepared all plugins
2026-08-28 13:51:26.634+0000 [id=63] INFO Jenkins.InitReactorRunner$1#onAttained: Started all plugins
2026-08-28 13:51:26.634+0000 [id=59] INFO Jenkins.InitReactorRunner$1#onAttained: Augmented all extensions
2026-08-28 13:51:26.711+0000 [id=50] INFO Jenkins.InitReactorRunner$1#onAttained: System config loaded
2026-08-28 13:51:26.711+0000 [id=50] INFO Jenkins.InitReactorRunner$1#onAttained: System config adapted
2026-08-28 13:51:26.711+0000 [id=50] INFO Jenkins.InitReactorRunner$1#onAttained: Loaded all jobs
2026-08-28 13:51:26.712+0000 [id=50] INFO Jenkins.InitReactorRunner$1#onAttained: Configuration for all jobs updated
2026-08-28 13:51:26.723+0000 [id=80] INFO Hudson.Util.Retrier#start: Attempt #1 to do the action check updates server
2026-08-28 13:51:26.706+0000 [id=56] INFO Jenkins.install.SetupWizard#init:
[LF]>
[LF]> *****
[LF]> *****
[LF]> *****
[LF]> *****
[LF]> Jenkins initial setup is required. An admin user has been created and a password generated.
[LF]> Please use the following password to proceed to installation:
[LF]>
[LF]> f28cc76edc0c4192be68a58f1b405a10
[LF]>
[LF]> This may also be found at: /var/jenkins_home/secrets/initialAdminPassword
[LF]>
[LF]> *****
[LF]> *****
[LF]> *****
2026-08-28 13:51:28.054+0000 [id=56] INFO Jenkins.InitReactorRunner$1#onAttained: Completed initialization
2026-08-28 13:51:28.061+0000 [id=40] INFO Hudson.Lifecycle.Lifecycle#onReady: Jenkins is fully up and running
2026-08-28 13:51:29.521+0000 [id=80] INFO h.m.DownloadService$Downloadable#load: Obtained the updated data file for Hudson.Tasks.Maven.MavenInstaller
2026-08-28 13:51:29.521+0000 [id=80] INFO Hudson.Util.Retrier#start: Performed the action check updates server successfully at the attempt #1
```

Démarrage

Débloquer Jenkins

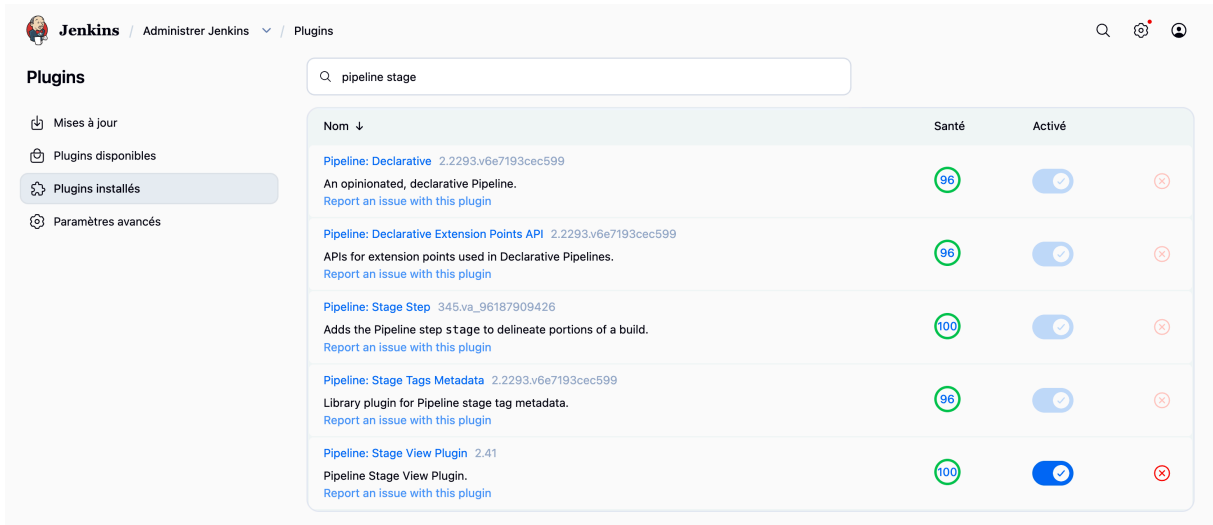
Pour être sûr que Jenkins soit configuré de façon sécurisée par un administrateur, un mot de passe a été généré dans le fichier de logs ([où le trouver](#)) ainsi que dans ce fichier sur le serveur :

`/var/jenkins_home/secrets/initialAdminPassword`

Veuillez copier le mot de passe depuis un des 2 endroits et le coller ci-dessous.

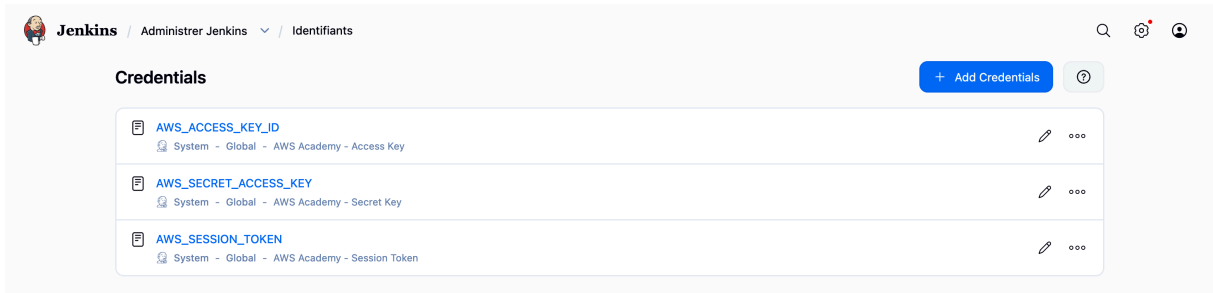
Mot de passe administrateur

Continuer



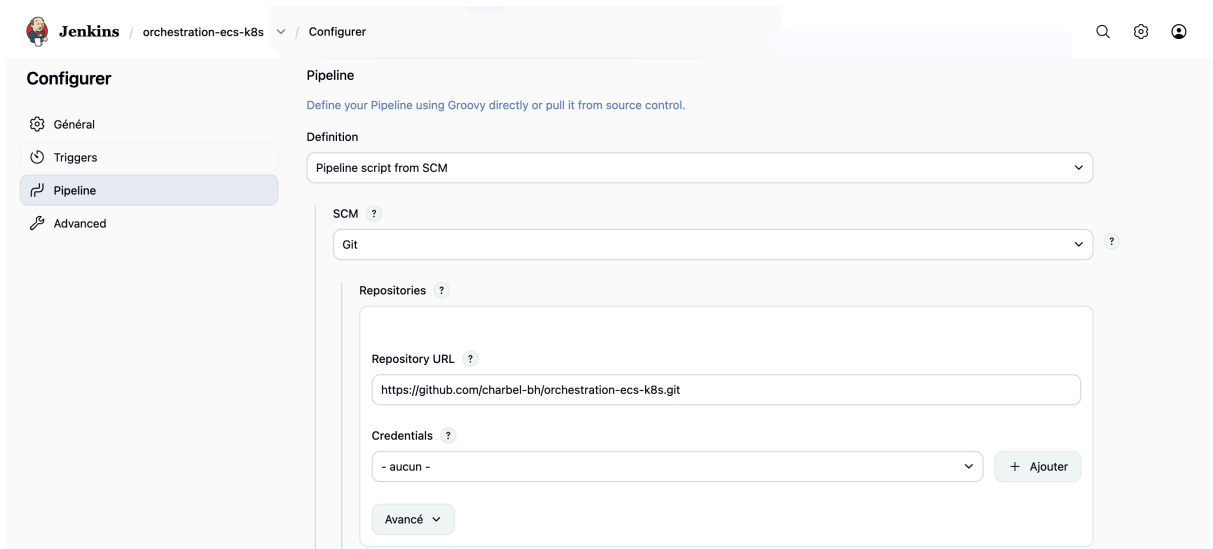
The Jenkins Plugins page shows a list of installed and available plugins. The search bar contains "pipeline stage". The table lists the following plugins:

Nom ↓	Santé	Activé
Pipeline: Declarative 2.2293.v6e7193cec599 An opinionated, declarative Pipeline. Report an issue with this plugin	96	☒
Pipeline: Declarative Extension Points API 2.2293.v6e7193cec599 APIs for extension points used in Declarative Pipelines. Report an issue with this plugin	96	☒
Pipeline: Stage Step 345.va_96187909426 Adds the Pipeline step stage to delineate portions of a build. Report an issue with this plugin	100	☒
Pipeline: Stage Tags Metadata 2.2293.v6e7193cec599 Library plugin for Pipeline stage tag metadata. Report an issue with this plugin	96	☒
Pipeline: Stage View Plugin 2.41 Pipeline Stage View Plugin. Report an issue with this plugin	100	☒



The Jenkins Credentials page shows a list of credentials. The search bar is empty. The table lists the following credentials:

Nom	Type	Id	Id
AWS_ACCESS_KEY_ID	System - Global - AWS Academy - Access Key		
AWS_SECRET_ACCESS_KEY	System - Global - AWS Academy - Secret Key		
AWS_SESSION_TOKEN	System - Global - AWS Academy - Session Token		



The Jenkins Pipeline Configuration page shows the configuration for a pipeline. The search bar is empty. The configuration is as follows:

Pipeline
Define your Pipeline using Groovy directly or pull it from source control.

Definition
Pipeline script from SCM

SCM ?
Git

Repositories ?

Repository URL ?
<https://github.com/charbel-bh/orchestration-ecs-k8s.git>

Credentials ?
- aucun - + Ajouter

Avancé ▾

Jenkins

orchestration-ecs-k8s

Configurer

Configurer

Général

Triggers

Pipeline

Advanced

Branches to build ?

Branch Specifier (blank for 'any') ?

*/main

+ Add Branch

Navigateur de la base de code ?

(Auto)

Additional Behaviours

+ Ajouter

Script Path ?

jenkins/Jenkinsfile

☒ Lightweight checkout ?

Stage View

		Declarative: Checkout SCM	Checkout	Validate	Import existing resources	Plan	Approve	Apply	Verify	Declarative: Post Actions
Average stage times:		564ms	465ms	21s	23s	6s	16ms	17ms	19ms	305ms
#4	août 28 19:44 No Changes	442ms	447ms	21s	37s	12s	14ms (paused for 48s)			

Appliquer ECS + Kubernetes

Apply

Abort

orchestration-ecs-k8s

Ajouter une description

Last Successful Artifacts

tfplan.txt 857 B view

Stage View

		Declarative: Checkout SCM	Checkout	Validate	Import existing resources	Plan	Approve	Apply	Verify	Declarative: Post Actions
Average stage times: (full run time: ~1min 49s)		638ms	461ms	21s	28s	8s	29ms	4s	375ms	300ms
#5	août 28 19:53 1 commit	788ms	454ms	22s	39s	12s	32ms (paused for 25s)	6s	1s	301ms

Partie 5 — Restitution

Le pipeline Jenkins a été créé sous le nom **orchestration-ecs-k8s**. Il récupère le **Jenkinsfile** depuis GitHub, initialise Terraform, valide la syntaxe et produit le plan d'exécution. L'artefact **tfplan.txt** est archivé par Jenkins, ce qui fournit une trace du plan lisible après le build. Le build est ensuite suspendu à l'étape **Approve**, jusqu'à une action manuelle de l'opérateur. Après approbation, Terraform applique le plan puis lance les vérifications finales.

Difficultés rencontrées et solutions

Difficulté	Cause identifiée	Solution appliquée
Échec de terraform plan avec No valid credential sources found	Les credentials AWS Academy n'étaient pas injectés dans l'environnement Jenkins	Création de trois credentials Jenkins de type Secret text et injection via les variables AWS standards
Échec de l'import avec aws: not found	AWS CLI absente du conteneur Jenkins	Installation d'AWS CLI dans le conteneur Jenkins ; l'outil peut ensuite être intégré à une image Jenkins personnalisée pour une persistance durable
Erreurs RepositoryAlreadyExistsException, InvalidGroup.Duplicate et already exists	Des ressources AWS et Kubernetes avaient été créées avant que Jenkins ne possède leur state Terraform	Ajout d'une étape d'import conditionnelle avant terraform plan
Échec lors de la création du service ECS, de l'Ingress et du HPA	Ces objets existaient mais n'avaient pas encore été importés dans le state Jenkins	Extension de l'étape d'import aux ressources ECS Service, Kubernetes Ingress et Kubernetes HPA

L'étape d'import a permis de migrer proprement les ressources existantes vers le state utilisé par Jenkins. Une fois toutes les ressources connues du state, les relances de pipeline deviennent idempotentes : Terraform ne doit appliquer que les différences réellement observées.

Sécurité, résilience et limites

Mesures mises en place

Les secrets AWS ne sont jamais versionnés dans Git : ils sont stockés dans Jenkins sous forme de credentials, puis injectés uniquement pendant l'exécution du pipeline. Le service ECS s'appuie sur un Security Group dédié et sur des images taguées explicitement, jamais en **latest**. Sa résilience est assurée par le scheduler ECS, qui relance automatiquement les tâches défailtantes. Côté Kubernetes, le Deployment utilise plusieurs réplicas et un HorizontalPodAutoscaler adapte leur nombre à la charge. L'étape **Approve** du pipeline Jenkins garantit qu'aucune modification n'est appliquée sans validation humaine, et le plan Terraform est archivé avant chaque application pour garder une trace des changements.

Limites identifiées et améliorations

Le state Terraform utilisé pendant le projet est local au workspace Jenkins. Cette solution est suffisante pour le prototype, mais elle n'est pas optimale pour un usage multi-utilisateur ou de production. Une amélioration consisterait à employer un backend distant, par exemple S3 avec mécanisme de verrouillage, afin de centraliser et protéger le state. L'installation d'AWS CLI directement dans le conteneur Jenkins est également une solution fonctionnelle mais non persistante lors d'une recréation du conteneur. Une image Docker Jenkins personnalisée devrait intégrer dès sa construction Terraform, kubectl et AWS CLI. Enfin, les mesures de sécurité Kubernetes peuvent être renforcées avec une **NetworkPolicy** et une politique d'admission telle que Kyverno interdisant le tag **latest**. Ces contrôles doivent être ajoutés si leur mise en œuvre est demandée lors de la démonstration.

Conclusion

Le projet a permis de mettre en œuvre une chaîne de déploiement unifiée couvrant ECS Fargate et Kubernetes Minikube. Terraform centralise la définition des ressources, tandis que Jenkins assure l'exécution contrôlée du cycle **validate** → **plan** → **approbation** → **apply** → **vérification**. La réussite du build Jenkins final confirme que les deux environnements sont pilotés par la même chaîne d'automatisation. Elles illustrent l'importance de la gestion des secrets, de la reproductibilité des environnements CI/CD et du state Terraform dans un contexte d'industrialisation.